

# From GPT to Chatbot

How a text predictor learns to be an assistant · video 2:18

This finale answers one question: how does a next-token predictor become a helpful assistant? It is trained in stages: **pretraining** on internet text, **supervised fine-tuning** on example conversations, then learning from **preferences** and other feedback. The machine never changes; only the **data** and the **training signal** do.

## How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

## Concepts in this lesson

| CONCEPT                              | VIDEO SEGMENT | QUESTIONS | MASTERED                 |
|--------------------------------------|---------------|-----------|--------------------------|
| 1. Pretraining makes a base model    | ▶ 0:08-0:31   | 4         | <input type="checkbox"/> |
| 2. Supervised fine-tuning            | ▶ 0:31-0:47   | 4         | <input type="checkbox"/> |
| 3. A chat is one long document       | ▶ 0:47-1:00   | 3         | <input type="checkbox"/> |
| 4. Learning from preferences         | ▶ 1:00-1:11   | 4         | <input type="checkbox"/> |
| 5. RLHF and DPO                      | ▶ 1:11-1:29   | 4         | <input type="checkbox"/> |
| 6. AI feedback and checkable answers | ▶ 1:29-1:40   | 4         | <input type="checkbox"/> |
| 7. Same machine, different data      | ▶ 1:40-2:11   | 4         | <input type="checkbox"/> |

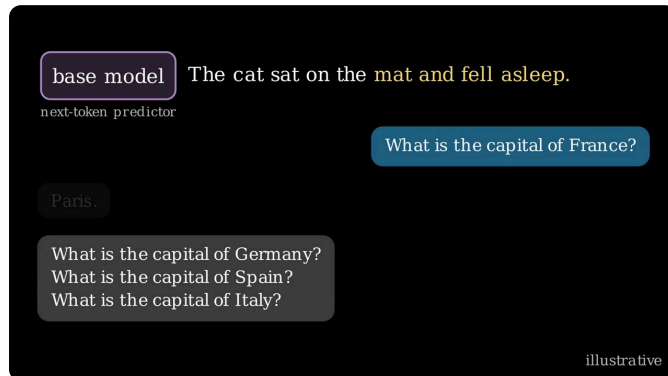
**Before you start:** this is the series finale, so it builds on the whole series: the generation loop (episode 1), the parts of the transformer (episodes 2–10) and training with the next-token loss (episode 11). Foundations F06 (Probability and Sampling) helps with the idea of a model's output as probabilities.

## CONCEPT 1 OF 7

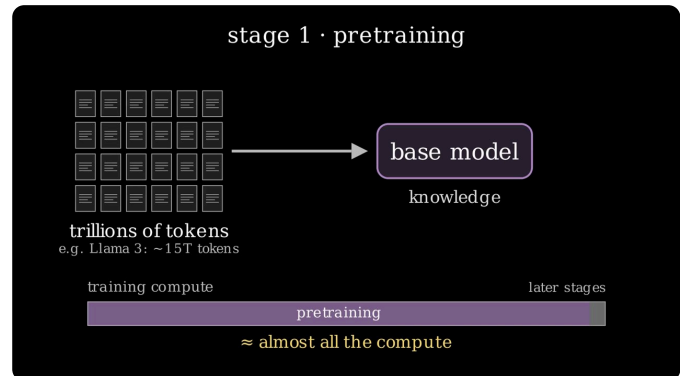
# Pretraining makes a base model

▶ Video 0:08-0:31

A model trained only to predict the next token of internet text is called a **base model**. It is a powerful **autocomplete**. Ask it a question and it might answer. Or it might continue with three more questions, as if it were writing a quiz: to a base model, your question is just text to continue.



▶ 0:22 A base model is an autocomplete: asked a question, it may answer, or carry on with more questions, like a quiz (illustrative).



▶ 0:30 Stage 1, pretraining: trillions of tokens of text, almost all the compute, and the model's knowledge.

This first stage is called **pretraining**. It is where almost all the computing goes, and where the **knowledge** comes from: **trillions of tokens** of text.

### KEY IDEA

Pretraining gives a **base model**: lots of knowledge, but its only habit is to continue text. It is not yet a helpful assistant.

### Check 1: Pretraining makes a base model

Answer all 4 questions before looking at the answers. Pass mark: all correct.

#### 1.1 MULTIPLE CHOICE

You type *"What is the capital of France?"* into a base model. What can happen?

- A. It always answers "Paris."
- B. It might answer, or it might continue with more questions, like a quiz
- C. It gives an error: base models cannot read questions
- D. It refuses, because it was never trained on questions

#### 1.2 TRUE OR FALSE

"Most of the training compute goes into the later stages that turn a base model into a chatbot."

True ☐ False ☐

#### 1.3 SHORT ANSWER

Where does a chatbot's knowledge mostly come from?

.....

#### 1.4 SHORT ANSWER

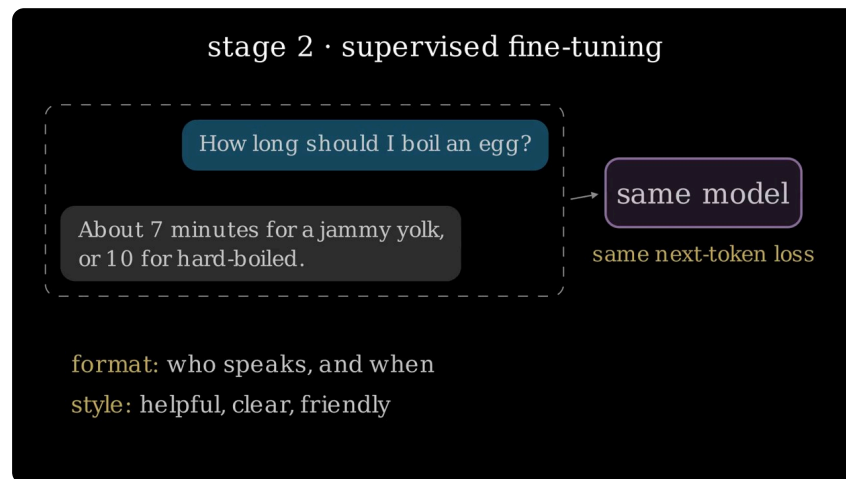
A new case: you want a base model to tell you the capital of France. How could you write the input so that simply continuing it gives the answer?

.....

## Supervised fine-tuning

▶ Video 0:31-0:47

Stage two is **supervised fine-tuning** (SFT). We keep training the **same model**, with the **same next-token loss** (episode 11), but now on examples of **conversations**: a user asks, and an assistant answers helpfully. From them the model learns the **format** (who speaks, and when) and the **style** (helpful, clear, friendly).



▶ 0:47 Stage 2: same model, same loss, example conversations.

### KEY IDEA

SFT = same model, same loss, **new data**: example conversations teach format and style.

### Check 2: Supervised fine-tuning

Answer all 4 questions before looking at the answers. Pass mark: all correct.

#### 2.1 MULTIPLE CHOICE

What changes when we move from pretraining to supervised fine-tuning?

- A. The model's architecture
- B. The loss function
- C. The training data
- D. The number of layers

#### 2.2 TRUE OR FALSE

"Supervised fine-tuning starts again from a new, random model."

True ☐ False ☐

#### 2.3 MULTIPLE CHOICE

Which of these is an SFT training example?

- A. A web page about cats
- B. A user's question followed by a helpful assistant answer
- C. Two answers, with a person's pick of the better one
- D. A math problem whose answer can be checked

#### 2.4 SHORT ANSWER

Name the two things the model learns from SFT examples, with a few words on each.

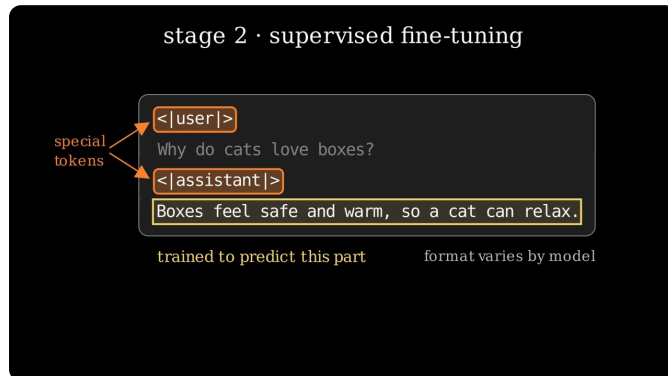
.....

## CONCEPT 3 OF 7

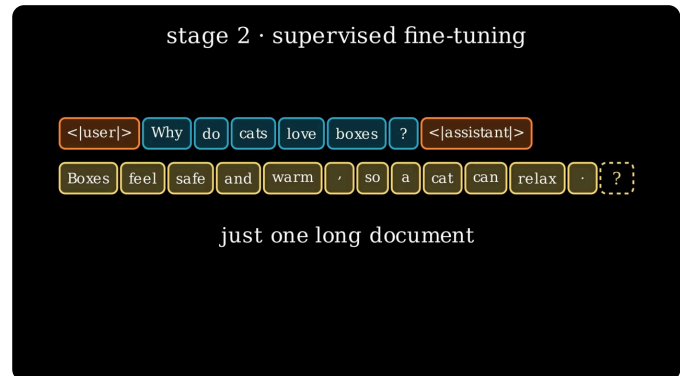
# A chat is one long document

▶ Video 0:47-1:00

Conversations are turned into text, with **special tokens** that mark who is speaking, like `<|user|>` and `<|assistant|>` in the video (the exact format varies by model). The model is trained to predict the **assistant's turns**.



▶ 0:56 A chat template: special tokens mark who is speaking. The model is trained to predict the assistant's part.



▶ 1:00 To the model, the whole chat is one long sequence of tokens to continue.

To the model, a chat is still **just one long document to continue**, with the same next-token prediction as always.

### KEY IDEA

A chat template turns a conversation into one sequence of tokens, with special tokens marking the speakers. The model learns to predict the **assistant's part**.

### Check 3: A chat is one long document

Answer all 3 questions before looking at the answers. Pass mark: all correct.

#### 3.1 MULTIPLE CHOICE

In the video's template, what are `<|user|>` and `<|assistant|>`?

- A. Ordinary words the model read on the internet
- B. Special tokens that mark who is speaking
- C. Comments that are deleted before training
- D. The names of two separate models

#### 3.2 TRUE OR FALSE

"A chatbot reads a conversation in a completely different way from how a base model reads text."

True ☐ False ☐

### 3.3 HANDS-ON CODE

**Try it yourself.** The code below turns the video's chat into tokens with a toy template (one token per word) and marks which tokens the model is trained to predict. (a) What do the two `print`s show? (b) Add a second exchange to `chat`: the user asks *"And dogs ?"* and the assistant answers *"Dogs prefer a warm bed ."* What are the two numbers now? (c) After training, a user asks a new question. With which token should the text given to the model end, so that its continuation is the assistant's answer?

```
chat = [("user", "Why do cats love boxes ?"),
        ("assistant", "Boxes feel safe and warm , so a cat can relax .")]

def to_tokens(chat):
    tokens, train = [], []
    for role, text in chat:
        tokens.append(f"<|{role}|>")           # special token: who speaks
        train.append(False)
        for word in text.split():               # toy tokenizer: one token per word
            tokens.append(word)
            train.append(role == "assistant")   # predict only the assistant's turn
    return tokens, train

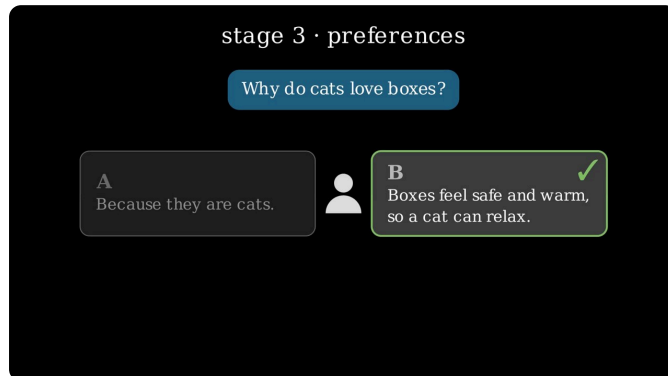
tokens, train = to_tokens(chat)
print(len(tokens), sum(train))
print([t for t, keep in zip(tokens, train) if keep][:3])
```

## CONCEPT 4 OF 7

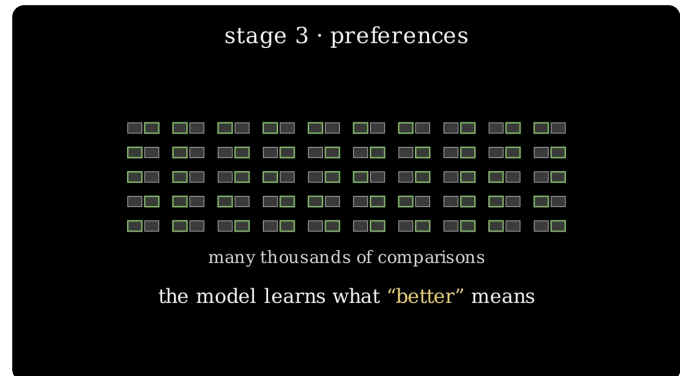
# Learning from preferences

▶ Video 1:00-1:11

Stage three teaches **judgment**. People compare **two answers from the model** to the same prompt, and **pick the better one**. In the video both answers have the right format, but only one is really helpful.



▶ 1:06 Two answers from the model to the same prompt. A person picks the better one.



▶ 1:10 Many thousands of comparisons teach the model what "better" means.

From many thousands of these comparisons, the model learns what **"better"** means.

### KEY IDEA

A preference example is a prompt, **two of the model's answers**, and **which one a person preferred**. Many thousands of them teach judgment.

### Check 4: Learning from preferences

Answer all 4 questions before looking at the answers. Pass mark: all correct.

#### 4.1 MULTIPLE CHOICE

What does one preference example contain?

- A. A prompt and one ideal answer written by a person
- B. A prompt, two answers from the model, and which one a person preferred
- C. A single answer with a grammar check
- D. A list of words the model must never use

#### 4.2 TRUE OR FALSE

"In the preference stage, people write the ideal answers themselves, and the model learns to copy them."

True ☐ False ☐

#### 4.3 PUT IN ORDER

Put the three training stages in order: *preferences* · *supervised fine-tuning* · *pretraining*.

#### 4.4 SHORT ANSWER

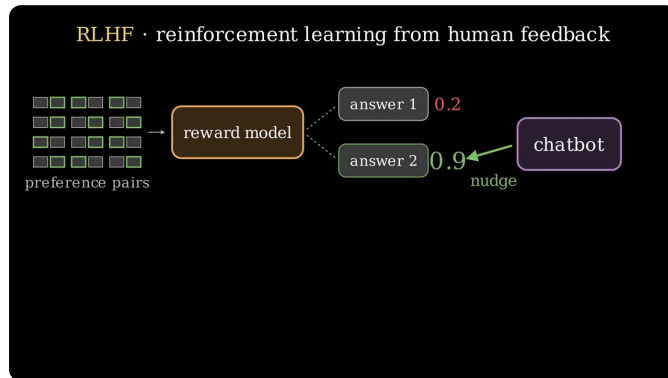
Both answers to "Why do cats love boxes?" are in the right format. So what does a person's pick of answer B teach the model?

## CONCEPT 5 OF 7

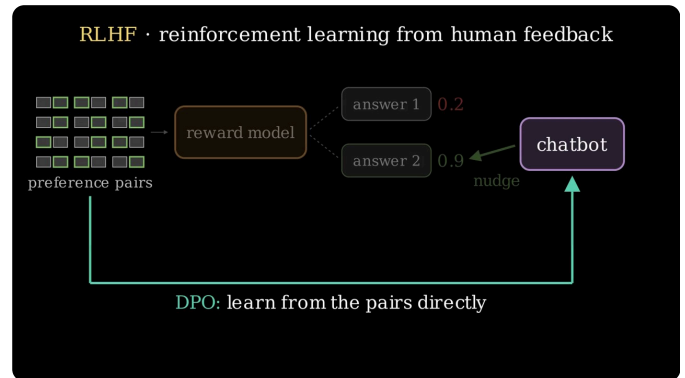
# RLHF and DPO

▶ Video 1:11-1:29

One way to use the comparisons is **RLHF**: reinforcement learning from human feedback. First, train a **reward model** to predict which answers people prefer; it gives each answer a score. Then **nudge the chatbot** toward answers that score higher. A simpler method, called **DPO**, learns from the preference pairs **directly**, without the reward model in between.



▶ 1:23 RLHF: a reward model learns to score answers the way people would; the chatbot is nudged toward higher scores.



▶ 1:29 DPO: skip the reward model and learn from the preference pairs directly.

### KEY IDEA

Same data, two routes. **RLHF**: pairs → reward model → nudge the chatbot toward higher scores. **DPO**: pairs → chatbot, directly.

### Check 5: RLHF and DPO

Answer all 4 questions before looking at the answers. Pass mark: all correct.

#### 5.1 PUT IN ORDER

Put the steps of RLHF in order: *nudge the chatbot toward higher scores · collect preference pairs · the reward model scores the chatbot's answers · train a reward model.*

#### 5.2 MULTIPLE CHOICE

The reward model scores two of the chatbot's answers 0.2 and 0.9. What does RLHF do with this?

- A. Nothing: the scores are only for people to read
- B. Nudges the chatbot toward answers like the 0.9 one
- C. Nudges the chatbot toward the 0.2 one
- D. Uses the reward model as the chatbot from now on

#### 5.3 TRUE OR FALSE

"DPO first trains a reward model, then uses it to nudge the chatbot."

True ☐ False ☐

#### 5.4 SHORT ANSWER

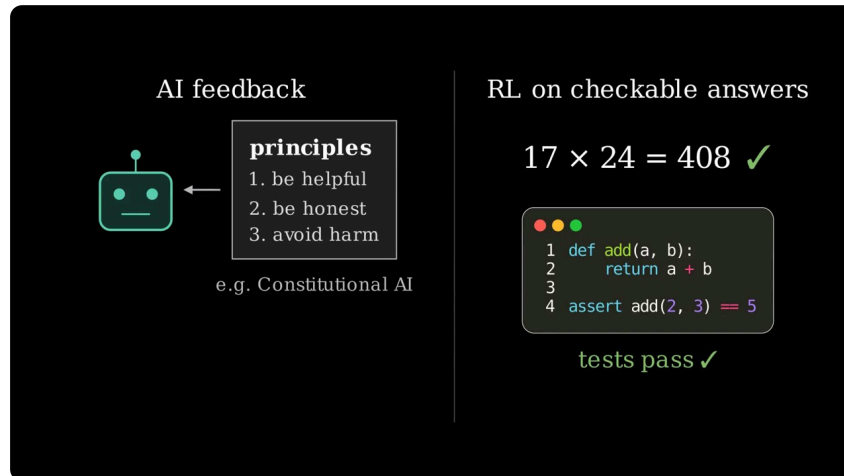
What do RLHF and DPO have in common, and what is the main difference?

## CONCEPT 6 OF 7

# AI feedback and checkable answers

▶ Video 1:29-1:40

People are not the only source of feedback. Many labs also use **AI feedback**, guided by **written principles** (in the video: be helpful, be honest, avoid harm), and **reinforcement learning on problems with checkable answers**, like math and code:  $17 \times 24 = 408$  can be checked, and code can be run against its tests.



▶ 1:39 Other signals: AI feedback, and RL on checkable answers.

### KEY IDEA

The training signal can come from people, from an AI guided by written principles, or, for problems like math and code, from **checking the answer**.

### Check 6: AI feedback and checkable answers

Answer all 4 questions before looking at the answers. Pass mark: all correct.

#### 6.1 MULTIPLE CHOICE

Which task is best suited to reinforcement learning on checkable answers?

- A. Writing a poem about autumn
- B. Working out  $17 \times 24$
- C. Choosing a friendly tone for an email
- D. Describing the mood of a painting

#### 6.2 TRUE OR FALSE

"With AI feedback, an AI gives the feedback, guided by written principles."

True ☐ False ☐

#### 6.3 SHORT ANSWER

A model writes a function `add(a, b)`. How can its answer be checked without a person?

.....

#### 6.4 SHORT ANSWER

A new case: a model's `add` always returns 5. Does it pass the video's test? What does that tell you?

.....

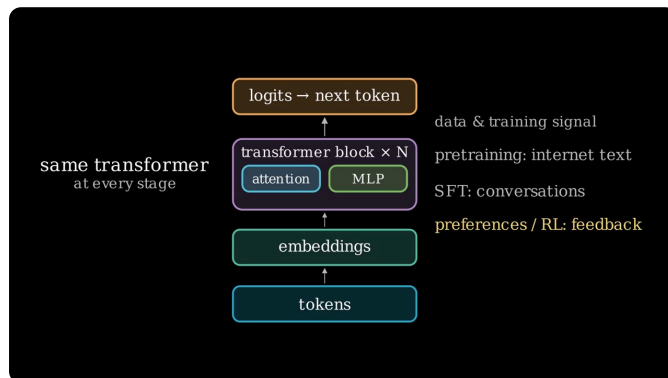


## CONCEPT 7 OF 7

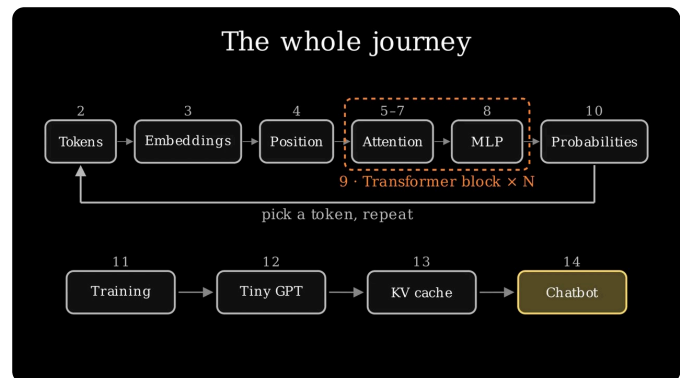
# Same machine, different data

▶ Video 1:40-2:11

Through all of this, the **machinery doesn't change**: the same transformer, with the same embeddings, attention and next-token prediction. Only the **data** and the **training signal** change: internet text, then conversations, then feedback. And that is the whole series: text becomes **tokens**, tokens become **vectors with positions**, **attention** lets them talk and **MLPs** let them think, stacked in **blocks**; the final vector becomes **probabilities**, we pick a token and repeat. Training shapes every number, and fine-tuning turns it into an assistant.



▶ 1:50 The same transformer at every stage.



▶ 2:10 The whole journey, from tokens to a chatbot.

### KEY IDEA

A chatbot is the same next-token predictor you met in every episode. What makes it an assistant is **the data and the training signal**.

### Check 7: Same machine, different data

Answer all 4 questions before looking at the answers. Pass mark: all correct.

#### 7.1 MULTIPLE CHOICE

What is different between a base model and the chatbot made from it?

- A. A new kind of network architecture
- B. The data and the training signal it was trained with
- C. Attention is replaced by a reward model
- D. It no longer predicts the next token

#### 7.2 SHORT ANSWER

For each stage, name the data it trains on: (a) pretraining, (b) supervised fine-tuning, (c) preferences and RL.

#### 7.3 PUT IN ORDER

Put in order: *probabilities · tokens · transformer blocks · pick a token · embeddings with positions*.

#### 7.4 TRUE OR FALSE

"A chatbot still writes its reply one token at a time, with episode 1's loop."

True ☐ False ☐

## ANSWERS

# Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

### 1. Pretraining makes a base model · rewatch ▶ 0:08-0:31

**1.1 B.** A base model only continues text, and a list of questions is a perfectly plausible continuation.

**1.2 False.** Almost all the computing goes into pretraining.

**1.3 Pretraining, on trillions of tokens of text.** The later stages teach format, style and judgment; the knowledge was learned in stage 1.

**1.4 Write the start of the answer, e.g. “The capital of France is”.** A base model continues text, and the most likely continuation of that sentence is “Paris”.

### 2. Supervised fine-tuning · rewatch ▶ 0:31-0:47

**2.1 C.** Same model, same next-token loss; only the data changes, to example conversations.

**2.2 False.** We keep training the same model, so everything it learned in pretraining is kept.

**2.3 B.** A is pretraining data, C is preference data (stage 3), and D is for reinforcement learning on checkable answers (concept 6).

**2.4 Format and style.** Format: who speaks, and when. Style: helpful, clear, friendly.

### 3. A chat is one long document · rewatch ▶ 0:47-1:00

**3.1 B.** They are special tokens added to the text so the model can tell the turns apart.

**3.2 False.** To the model, a chat is just one long document, and it predicts the next token as always.

**3.3 (a) 20 12 and ['Boxes', 'feel', 'safe'] · (b) 31 18 · (c) <|assistant|>** (a) 20 tokens: 7 for the user's turn (<|user|> + 6 words) and 13 for the assistant's (<|assistant|> + 12). Only the assistant's 12 words are trained, starting *Boxes feel safe*. (b) The new exchange adds 1 + 3 user tokens and 1 + 6 assistant tokens: 20 + 11 = 31 tokens, and 12 + 6 = 18 trained. (c) Give <|user|>, the question, then <|assistant|>. The chat is one document, and after that token the natural continuation is an assistant's answer: exactly what the model was trained to predict.

### 4. Learning from preferences · rewatch ▶ 1:00-1:11

**4.1 B.** People compare two of the model's own answers and pick the better one.

**4.2 False.** People only compare and pick. Learning from written example answers is closer to SFT.

**4.3 pretraining → supervised fine-tuning → preferences.** Knowledge first, then the format and style of an assistant, then judgment.

**4.4 Judgment: which of two well-formed answers is better.** SFT already taught the format; comparisons teach what “better” means.

## 5. RLHF and DPO · rewatch ► 1:11-1:29

**5.1 collect preference pairs → train a reward model → the reward model scores the chatbot's answers → nudge the chatbot toward higher scores.** The reward model must learn from people's picks before it can score new answers.

**5.2 B.** The chatbot is nudged toward answers that score higher, i.e. that people would probably prefer.

**5.3 False.** That is RLHF. DPO learns from the preference pairs directly.

**5.4 Both learn from the same preference pairs.** RLHF goes through a reward model and nudges the chatbot toward higher scores; DPO learns from the pairs directly, which is simpler.

## 6. AI feedback and checkable answers · rewatch ► 1:29-1:40

**6.1 B.** A calculation has one right answer that can be checked. The others are matters of judgment.

**6.2 True.** That is how the video describes it (one example is Constitutional AI).

**6.3 Run tests on it, like `assert add(2, 3) == 5`.** If the tests pass, the answer counts as good, as in the video's "tests pass ✓".

**6.4 Yes, it passes, although it is wrong.** `add(2, 3)` happens to be 5. A check is only as good as its tests; more tests (like `add(1, 1) == 2`) would catch it.

## 7. Same machine, different data · rewatch ► 1:40-2:11

**7.1 B.** Same transformer, same next-token prediction; only the training changes.

**7.2 (a) Internet text. (b) Conversations. (c) Feedback.** Feedback from people's comparisons, from an AI guided by principles, or from checking answers.

**7.3 tokens → embeddings with positions → transformer blocks → probabilities → pick a token (then repeat).** This is the data flow of episodes 2–10, closed into the loop of episode 1.

**7.4 True.** Nothing about generation changes: the reply is still produced token by token, as a continuation of the chat document.